

Đánh giá kiến trúc MLOps ở giai đoạn hiện tại

Hiện trạng, khoảng trống và lộ trình Degradation-Aware Continuous Training

August 28, 2026

Abstract

Báo cáo này tiến hành đánh giá toàn diện repository MLOps PaaS dựa trên khung kiến trúc tham chiếu chuẩn mực của Amou Najafabadi *et al.* [1]—một nghiên cứu tổng hợp (systematic mapping study) từ 93 công trình khoa học, bao quát 39 thành phần kiến trúc, 56 bước quy trình và 15 vai trò stakeholder. Kết quả phân tích mã nguồn thực tế cho thấy hệ thống đã thiết lập nền tảng vững chắc cho một nền tảng vận hành mô hình (MLOps PaaS): control plane phân tách đa tenant, registry quản lý phiên bản bất biến, hạ tầng huấn luyện đa backend (Docker/Argo/Kubeflow), dịch vụ đóng gói tự động (model-packager), serving gateway và luồng thu thập production log phục vụ giám sát data drift.

Tuy nhiên, đối chiếu theo phân loại của nghiên cứu tham chiếu, hệ thống hiện dừng lại ở biến thể **V1 (Inference Service + Manual Retraining)**, chưa hình thành vòng lặp **Continuous Training (CT - biến thể V3)** khép kín. Các khoảng trống kiến trúc trọng yếu bao gồm: thiếu cơ sở dữ liệu phản hồi nhận thực tế (Feedback Database), chưa có cơ chế đánh giá hiệu năng suy giảm (performance degradation), thiếu công cụ so sánh tự động candidate–champion và chưa hỗ trợ điều phối lưu lượng shadow/canary. Trên cơ sở đó, báo cáo đề xuất hướng nghiên cứu và luận đề **“Degradation-Aware Adaptive Model Maintenance”** (Bảo trì mô hình thích ứng nhận thức suy giảm), phân tách tường minh ba lớp: phát hiện (detection), chẩn đoán nguyên nhân (diagnosis) và ra quyết định hành động (action selection). Trong kiến trúc này, tái huấn luyện (retraining) là một hành động có điều kiện kiểm chứng nghiêm ngặt thay vì phản xạ mặc định sau mỗi cảnh báo drift.

1 Phạm vi và phương pháp đánh giá

1.1 Khung kiến trúc tham chiếu

Nghiên cứu tham chiếu của Amou Najafabadi *et al.* [1] mô tả hệ thống MLOps thông qua hai chiều cạnh cấu trúc cốt lõi:

- **Cấu trúc tĩnh (Structure):** Gồm 39 thành phần chức năng được phân bổ trong 6 nhóm: (1) Data Curation, (2) Storage & Versioning, (3) ML Training, (4) CI/CD, (5) Inference, và (6) Infrastructure & Supporting Services.
- **Quy trình động (Process):** Gồm 56 bước quy trình bao quát trọn vẹn vòng đời từ thu thập dữ liệu, huấn luyện, kiểm thử, triển khai đến giám sát và bảo trì trong môi trường sản xuất.
- **Không gian biến thể (Variants):** Phân định 8 biến thể kiến trúc (V1–V8). Trong đó, nhóm cốt lõi V1–V4 áp dụng cho mô hình huấn luyện từ đầu (train from scratch), phân biệt bởi hình thức phục vụ dự đoán (Inference Service độc lập vs. Inference Engine Pool dùng chung) và phương thức tái huấn luyện (Manual vs. Continuous Training). Các biến thể V5–V8 tương ứng áp dụng cho mô hình nạp sẵn (pre-trained/fine-tuned).

Khung tham chiếu trên được sử dụng như một chuẩn mực đánh giá cấu trúc và phân tích khoảng trống (gap analysis), giúp định hình hướng phát triển học thuật thay vì xem xét như một danh mục kiểm tra cứng nhắc.

1.2 Phương pháp thẩm định mã nguồn

Đánh giá được thực hiện trực tiếp trên mã nguồn, schema cơ sở dữ liệu, đặc tả hạ tầng Kubernetes/Docker và các bộ kiểm thử tự động của repository. Các luồng nghiệp vụ chính được rà soát bao gồm: (1) upload/build artifacts, (2) training execution & tracking, (3) model serving & event stream, (4) model registry & deployment, và (5) drift monitoring workflow. Một thành phần được ghi nhận là “Thiếu” thể hiện rằng nền tảng chưa cung cấp contract hoặc module cài đặt sẵn trong hệ thống quản trị, không hàm ý người dùng bị giới hạn khi tự định nghĩa logic trong mã nguồn mô hình riêng.

2 Kết luận điều hành (Executive Summary)

Tổng thể mã nguồn xác nhận repository là một **nền tảng MLOps PaaS mạnh mẽ ở khâu vận hành và triển khai mô hình (Model Operations)**, nhưng chưa hoàn thiện năng lực **tự thích ứng và bảo trì mô hình khép kín (Closed-Loop Continuous Training)**. Điểm mạnh nổi bật nằm ở kiến trúc phân tầng rõ ràng (Control Plane vs. Data Plane vs. Execution Plane), tính bất biến của phiên bản mô hình, tính đa dạng của backend huấn luyện và kênh thu thập sự kiện suy luận. Ngược lại, điểm nghẽn lớn nhất nằm ở sự thiếu vắng vòng lặp phản hồi nhân thực tế, thiếu cơ chế giám sát chất lượng/hiệu năng suy giảm và chưa có trí tuệ ra quyết định (decision intelligence) giữa khâu cảnh báo và kích hoạt tái huấn luyện.

Table 1: Đánh giá tổng hợp năng lực hệ thống theo khung kiến trúc tham chiếu.

Trục kiến trúc	Mức độ	Nhận định tổng quan
Nền tảng & Phân tầng	7/10	Phân tách rõ ràng giữa Control, Data và Execution Plane; phân quyền tenant chuẩn mực.
Lưu trữ & Quản lý phiên bản	7/10	Quản lý artifact, digest image và snapshot huấn luyện bất biến; thiếu Feedback DB.
Huấn luyện & CI/CD	7/10	Hỗ trợ đa backend (Docker/Argo/Kubeflow), đóng gói tự động; thiếu Evaluation Gate tự động.
Suy luận & Quan sát	6/10	Gateway serving tốt, thu thập event stream ổn định; chưa đo lường performance thực tế.
Vòng đời bảo trì khép kín	4/10	Tự động kích hoạt DriftRun khi đủ mẫu; chưa có cơ chế chặn đoán nguyên nhân và CT policy.
Quản trị & Stakeholder	4/10	Có xác thực JWT/API-Key và audit log; chưa có luồng phê duyệt (approval) cho bảo trì mô hình.

Xác định biến thể kiến trúc hiện tại: Đối chiếu với phân loại của Amou Najafabadi *et al.* [1], hệ thống hiện đáp ứng trọn vẹn đặc trưng của **Biến thể V1 (Inference Service + Manual Retraining)**: dịch vụ model-packager đóng gói mô hình thành container độc lập, task deployment đẩy lên hạ tầng production, và model-server định tuyến phục vụ dự đoán. Việc tái huấn luyện hoàn toàn phụ thuộc vào thao tác thủ công của kỹ sư ML. Mục tiêu của đề tài là nghiên cứu và nâng cấp hệ thống từ biến thể V1 lên **Biến thể V3 (Continuous Training Pipeline)** có tích hợp cơ chế nhận thức suy giảm thích ứng.

3 Đối chiếu cấu trúc kiến trúc — Mapping 39 thành phần

Phần này ánh xạ chi tiết 39 thành phần chức năng của khung kiến trúc tham chiếu [1] với hiện trạng mã nguồn thực tế của repository, chia theo 6 nhóm chức năng. Sơ đồ Hình 1 mô tả mối quan hệ tương tác giữa 6 nhóm nền tảng và làm nổi bật vòng lặp ra quyết định (Decision Loop) đề xuất bổ sung.

3.1 Data Curation

Table 2: Ánh xạ nhóm Data Curation.

Thành phần	Mức độ	Bằng chứng trong mã nguồn	Khoảng trống kiến trúc / Hướng phát triển
Data Collector	Một phần	Workspace assets hỗ trợ upload mã/dữ liệu; TrainingJob nhận data_snapshot_uri.	Chưa có đường ống thu nạp dữ liệu tự động (automated data ingestion) từ nguồn bên ngoài.
Data Source	Một phần	Hỗ trợ nạp dataset qua workspace; asset tham chiếu (reference asset) cho drift.	Nền tảng chưa quản trị kết nối trực tiếp tới các nguồn dữ liệu ngoại vi (APIs, Streaming, DBs).
Data Preprocessor	Thiếu	Tiền xử lý dữ liệu được thực thi trực tiếp trong mã người dùng tại entry point.	Chưa có pipeline tiền xử lý và chuẩn hóa dữ liệu được quản lý ở cấp độ nền tảng (platform-managed).

Thành phần	Mức độ	Bằng chứng trong mã nguồn	Khoảng trống kiến trúc / Hướng phát triển
Data Quality Checker	Thiếu	Chưa có module hiện thực sẵn trong nền tảng.	Khoảng trống trọng yếu; cần thiết để ngăn chặn việc tái huấn luyện trên dữ liệu sai hỏng/lỗi schema.
Data Labeler	Thiếu	Chưa có luồng gán nhãn hoặc tích hợp hệ thống annotation.	Cần thiết cho vòng lặp tiếp nhận nhãn trễ (delayed-label feedback loop).

3.2 Storage & Versioning

Table 3: Ánh xạ nhóm Storage & Versioning.

Thành phần	Mức độ	Bằng chứng trong mã nguồn	Khoảng trống kiến trúc / Hướng phát triển
Data Catalog	Thiếu	Chưa có metadata catalog quản lý danh mục dataset tập trung.	Cần thiết nhằm truy xuất toàn diện nguồn gốc dữ liệu phục vụ mô hình (data → model lineage).
Dataset Repository	Một phần	S3 lưu trữ các snapshot dữ liệu huấn luyện (data_snapshot_uri); workspace assets.	Dữ liệu chưa được mô hình hóa thành thực thể phiên bản độc lập; lưu trữ dạng snapshot tĩnh.
Raw Data Store	Một phần	S3 lưu trữ raw/reference assets; production logs lưu trong PostgreSQL.	Chưa phân tách triệt để giữa tầng lưu trữ dữ liệu thô (raw), sơ chế (processed) và dữ liệu chuẩn (curated).
Feature Store	Thiếu	Chưa có module hiện thực trong hệ thống.	Các đặc trưng hiện được tính toán trực tiếp trong mã nguồn người dùng.
Code Repository	Có	Quản lý code_snapshot_uri bất biến trong TrainingJob; liên kết Git repo.	Đã đáp ứng trọn vẹn yêu cầu quản lý phiên bản mã nguồn huấn luyện.
Model Repository	Có	ModelVersion + ModelArtifact + ModelMetric trong ứng dụng registry; phiên bản bất biến, metrics/params summary, artifacts.	Đạt chuẩn mực cao; đây là năng lực nền tảng cốt lõi của hệ thống.
Artifact Repository	Có	S3 + thực thể ModelArtifact lưu trữ container image và artifacts theo phiên bản; checksum cố định.	Đã đáp ứng tốt cho biến thể V1; cần mở rộng liên kết với công thẩm định candidate.
ML Metadata Repository	Có	Tracking TrainingJob (JSON), TrainingOutput, TrainingJobEvent, RegistryEvent; tích hợp MLflow.	Đã hoàn thiện ở mức nền tảng; cần bổ sung thêm environment lineage và configuration lineage.

Thành phần	Mức độ	Bằng chứng trong mã nguồn	Khoảng trống kiến trúc / Hướng phát triển
Feedback Database	Thiếu	Schema PostgreSQL có các trường runtime nhưng chưa có Feedback API/Store cho ground truth.	Khoảng trống then chốt —cần xây dựng bảng Feedback liên kết trực tiếp với prediction_id.
Predictions Storage	Có	Bảng paas_production_logs lưu trữ log suy luận phân vùng theo tenant/project/version.	Đã có lưu trữ, nhưng cần bổ sung các trường runtime (confidence, latency, status) vào event stream.

3.3 ML Training

Table 4: Ảnh xạ nhóm ML Training.

Thành phần	Mức độ	Bằng chứng trong mã nguồn	Khoảng trống kiến trúc / Hướng phát triển
Feature Engineering Pipeline	Một phần	Biến đổi đặc trưng nằm trong mã người dùng; chưa có pipeline quản lý tập trung.	Cần chuẩn hóa biến đổi và phiên bản hóa đặc trưng khi logic feature ảnh hưởng tới CT.
ML Experiment Pipeline	Có	TrainingJob hỗ trợ entry point, snapshot mã/dữ liệu, tích hợp MLflow; Docker hoặc Argo/Kubeflow.	Nền tảng thực thi thử nghiệm hoạt động tin cậy và linh hoạt.
Continuous Training Pipeline	Thiếu	Chưa có luồng tự động hóa tái huấn luyện khép kín.	Trọng tâm nghiên cứu cốt lõi của đề tài.
Model Evaluator	Một phần	Trích xuất metrics qua MLflow/TrainingOutput; chưa có cổng thẩm định đối kháng tự động.	Cần xây dựng thực thể EvaluationGate phục vụ so sánh Champion–Candidate.
Fine-Tuning Module	Thiếu	Chưa hỗ trợ quy trình tinh chỉnh mô hình nạp sẵn chuyên biệt.	Thành phần mở rộng cho các biến thể V5–V8 trong tương lai.

3.4 CI/CD

Table 5: Ảnh xạ nhóm CI/CD.

Thành phần	Mức độ	Bằng chứng trong mã nguồn	Khoảng trống kiến trúc / Hướng phát triển
ML Component Builder	Có	Dịch vụ model-packager đóng gói container image từ training artifacts; theo dõi trạng thái, digest.	Đã hoàn thiện vững chắc cho quy trình đóng gói tự động.
ML Component Deployer	Có	Thực thể Deployment, Endpoint, Kustomize/GitOps manifests; Celery task orchestration.	Đã đáp ứng tốt việc triển khai containerized component lên hạ tầng.

Thành phần	Mức độ	Bằng chứng trong mã nguồn	Khoảng trống kiến trúc / Hướng phát triển
ML Model Deployer	Thiếu	Chưa hỗ trợ nạp mô hình động vào một Inference Engine Pool dùng chung.	Chỉ áp dụng khi mở rộng sang biến thể V2/V4.
ML Pipeline Builder	Thiếu	Chưa có module đóng gói toàn bộ pipeline huấn luyện thành pipeline artifact.	Cần thiết khi chuyển dịch lên biến thể V3 (Continuous Training).
ML Pipeline Deployer	Thiếu	Chưa có module triển khai và kích hoạt pipeline huấn luyện tự động định kỳ.	Cần thiết khi chuyển dịch lên biến thể V3 (Continuous Training).

3.5 Inference

Table 6: Ánh xạ nhóm Inference.

Thành phần	Mức độ	Bằng chứng trong mã nguồn	Khoảng trống kiến trúc / Hướng phát triển
Inference Service	Có	Model-server gateway + ML/DL serving workers; định tuyến theo phiên bản; xác thực JWT/API-Key; Prometheus metrics.	Nền tảng phục vụ suy luận hoạt động ổn định và tin cậy (đặc trưng V1).
Inference Engine Pool	Thiếu	Không sử dụng mô hình engine pool nạp động đa mô hình.	Không thuộc phạm vi kiến trúc hướng tới (V1/V3).
Runtime Model Monitor	Một phần	Prometheus theo dõi latency/request count; Redpanda event stream; Evidently tính toán data drift.	Còn thiếu: giám sát hiệu năng thực tế (F1/Recall trên nhãn trễ) và trôi dạt phân phối dự đoán.
Trigger	Một phần	Transactional Outbox tạo DriftRun tự động khi đủ ngưỡng số lượng bản ghi tích lũy.	Hiện chỉ kích hoạt tính toán drift, chưa kết nối tới luồng chẩn đoán hay quyết định CT.
Model Comparison Runner	Thiếu	Giao diện so sánh mô hình hiện là placeholder, chưa có engine so sánh đối kháng thực tế.	Khoảng trống trọng yếu —cần thiết để thẩm định candidate trước khi triển khai.
Decision Processor	Thiếu	Chưa có khối logic ra quyết định bảo trì mô hình.	Sẽ được hiện thực hóa qua module Chẩn đoán và RetrainingRequest.
XAI Module	Thiếu	Chưa tích hợp module giải thích mô hình (Explainable AI).	Thành phần mở rộng nghiên cứu, không thuộc phạm vi cốt lõi của đề tài.

3.6 Infrastructure & Supporting Services

Table 7: Ánh xạ nhóm Infrastructure & Supporting Services.

Thành phần	Mức độ	Bằng chứng trong mã nguồn	Khoảng trống kiến trúc / Hướng phát triển
Resource Manager	Một phần	TrainingJob quản lý cấu hình tài nguyên (vcpu, memory_mb, GPU); Kubernetes resource limits.	Còn thiếu: cơ chế quan sát chi phí tính toán (cost observability) và metrics GPU chi tiết.
Communication Middleware	Có	Redpanda (tương thích Kafka) truyền nhận sự kiện suy luận; EventOutbox; Celery + Redis cho hàng đợi tác vụ.	Đạt chuẩn mực cao về tính ổn định và khả năng mở rộng.
Container Orchestrator	Có	Docker Compose cho môi trường phát triển cục bộ; Kubernetes/K3s cho môi trường sản xuất.	Đã đáp ứng trọn vẹn yêu cầu điều phối container.
Workflow Orchestrator	Có	Celery xử lý tác vụ bất đồng bộ; Argo Workflows/Kubeflow điều phối huấn luyện quy mô lớn; GitOps.	Hệ thống điều phối quy trình hoạt động hoàn chỉnh và linh hoạt.
Log Manager	Có	Bảng paas_production_logs; nhật ký TrainingJobEvent; RegistryEvent; Transactional Outbox.	Hạ tầng quản lý log phân tầng và đảm bảo tính nhất quán dữ liệu.
API Gateway	Có	Model-server FastAPI gateway; Traefik reverse proxy; Django REST framework APIs tại Control Plane.	Đã hoàn thiện tốt khả năng định tuyến và kiểm soát truy cập an toàn.
MLOps User Interaction Manager	Một phần	Giao diện người dùng React/Vite; dashboard theo dõi drift; giao diện quản lý training/deployment.	Cần bổ sung dòng thời gian bằng chứng, hộp thư duyệt yêu cầu và giao diện so sánh mô hình.

3.7 Tổng hợp kết quả ánh xạ 39 thành phần

3.8 Nền tảng kiến trúc hiện hữu có giá trị kế thừa

- **Phân tách ranh giới các mặt phẳng (Planes Boundary):** Hệ thống phân định rõ ràng giữa Control Plane (điều phối vòng đời, xác thực đa tenant), Data Plane (phục vụ suy luận với độ trễ thấp) và Execution Plane (huấn luyện phi tập trung qua Docker hoặc Argo/Kubeflow). Trạng thái nghiệp vụ được bảo đảm tính nhất quán bền vững trên PostgreSQL và S3.
- **Tính bất biến của phiên bản mô hình (Version Immutability):** Mọi build thành công đều gắn liền với digest container image và artifact checksum cố định. Khái niệm Model Alias cho phép thăng cấp (promotion) hoặc thu hồi (rollback) linh hoạt mà không làm biến đổi lịch sử phiên bản.
- **Kênh truyền nhận sự kiện sản xuất an toàn (Production Event Pipeline):** Gateway truyền dữ liệu suy luận có phân quyền tenant/project sang Redpanda/Kafka; dịch vụ Consumer lưu vết vào bảng paas_production_logs và sử dụng Transactional Outbox pattern để tạo tín hiệu kích hoạt tính toán drift an toàn.
- **Quy trình giám sát trôi dạt dữ liệu (Drift Workflow):** Hỗ trợ liên kết model version với dữ liệu tham chiếu (reference dataset); Evidently tính toán chỉ số trôi dạt; callback cập nhật trạng thái DriftRun dựa trên UUID và idempotency key.
- **Hạ tầng huấn luyện linh hoạt:** Thực thể TrainingJob hỗ trợ cấu hình tài nguyên chi tiết (vCPU, RAM, GPU), cơ chế bảo mật capabilities-based và lưu vết bất biến mã nguồn (code_snapshot_uri) cùng dữ liệu

Table 8: Tổng hợp mức độ đáp ứng 39 thành phần chức năng theo từng nhóm.

Nhóm thành phần	Có	Một phần	Thiếu	Nhận định kiến trúc
Data Curation (5)	0	2	3	Nhóm yếu nhất; cần bổ sung Data Quality Checker và Feedback cho CT.
Storage & Versioning (10)	5	2	3	Model/Artifact Repository mạnh mẽ; thiếu Feedback Database liên kết.
ML Training (5)	1	2	2	ML Experiment Pipeline tốt; Continuous Training là trọng tâm phát triển.
CI/CD (5)	2	0	3	Đóng gói/triển khai tốt cho V1; thiếu pipeline builder/deployer cho V3.
Inference (7)	1	2	4	Serving gateway ổn định; thiếu Model Comparison Runner và performance monitoring.
Infrastructure & Supporting (7)	5	2	0	Hạ tầng nền tảng hoàn thiện; cần bổ sung giao diện tương tác nâng cao.
Tổng cộng (39)	14	10	15	Đánh giá bao quát đủ 39 thành phần nhằm minh bạch phạm vi và định vị rõ ranh giới nghiên cứu.

(data_snapshot_uri).

3.9 Phân tích các khoảng trống kiến trúc then chốt

Đối chiếu với yêu cầu của một hệ thống Continuous Training thích ứng, repository hiện tồn tại 6 khoảng trống cốt lõi:

1. **Hợp đồng dữ liệu sự kiện (Event Contract) chưa hoàn chỉnh:** Mặc dù schema consumer đã định nghĩa các trường `confidence`, `latency_ms`, `status_code` và `request_id`, nhưng gateway `model-server` hiện chỉ mới gửi dữ liệu `features` và `output` thô. Điểm số tự tin (`confidence`) và các thông số vận hành chưa được truyền tải vào event stream.
2. **Đứt gãy mối nối định danh dự đoán (Prediction ID Correlation):** Khóa định danh sự kiện `id` được sinh nội bộ tại gateway nhưng chưa được trả về cho phía client trong response. Do đó, client không có mã tham chiếu để gửi nhãn thực tế (`ground truth`) tương ứng sau này.
3. **Phạm vi giám sát dừng lại ở Data Drift:** Evidently hiện chỉ đo lường sự dịch chuyển phân phối dữ liệu đầu vào ($P(X)$), chưa hỗ trợ đánh giá chất lượng dữ liệu (`data quality checks`) cũng như chưa đo lường suy giảm hiệu năng thực tế ($P(Y|X)$, `F1`, `Precision`, `Recall`) do thiếu nhãn.
4. **Cơ chế kích hoạt thụ động (Passive Triggering):** Bảng `paas_automatic_drift_outbox` chỉ kích hoạt việc tạo một lượt chạy `DriftRun` khi tích lũy đủ số lượng bản ghi, hoàn toàn chưa có liên kết tự động tới luồng chẩn đoán hay yêu cầu tái huấn luyện.
5. **Quy trình đánh giá và thăng cấp thủ công:** Chức năng so sánh mô hình trên giao diện hiện chỉ hiển thị placeholder, thiếu một `Evaluation Gate` có thể tự động kiểm tra đối kháng (`champion vs. candidate`) trước khi quyết định triển khai.
6. **Chưa hỗ trợ phân tách lưu lượng (Traffic Splitting):** Gateway chỉ định tuyến toàn bộ 100% lưu lượng tới một phiên bản duy nhất, chưa hỗ trợ `Shadow Mode` (nhân bản request) hoặc `Canary Rollout` (chia tỉ lệ lưu lượng động).

4 Đối chiếu quy trình vòng đời (56 Process Steps)

Kết quả đối chiếu quy trình khẳng định một kết luận kiến trúc quan trọng: **hệ thống đã xây dựng hoàn chỉnh hạ tầng đường ống (Ingestion Plumbing), nhưng còn thiếu hoàn toàn khối trí tuệ ra quyết định (Adaptive Decision Loop)**. Việc chỉ gắn thêm một lệnh kích hoạt tái huấn luyện đơn giản sau khi phát hiện `data drift` sẽ không tạo nên một hệ thống CT tin cậy, bởi vì: (1) `data drift` không đồng nghĩa với suy giảm hiệu năng thực tế (có thể là `benign drift`), và (2) việc tự động huấn luyện lại trên dữ liệu bị lỗi hoặc nhiễu sẽ làm lặp lại hoặc

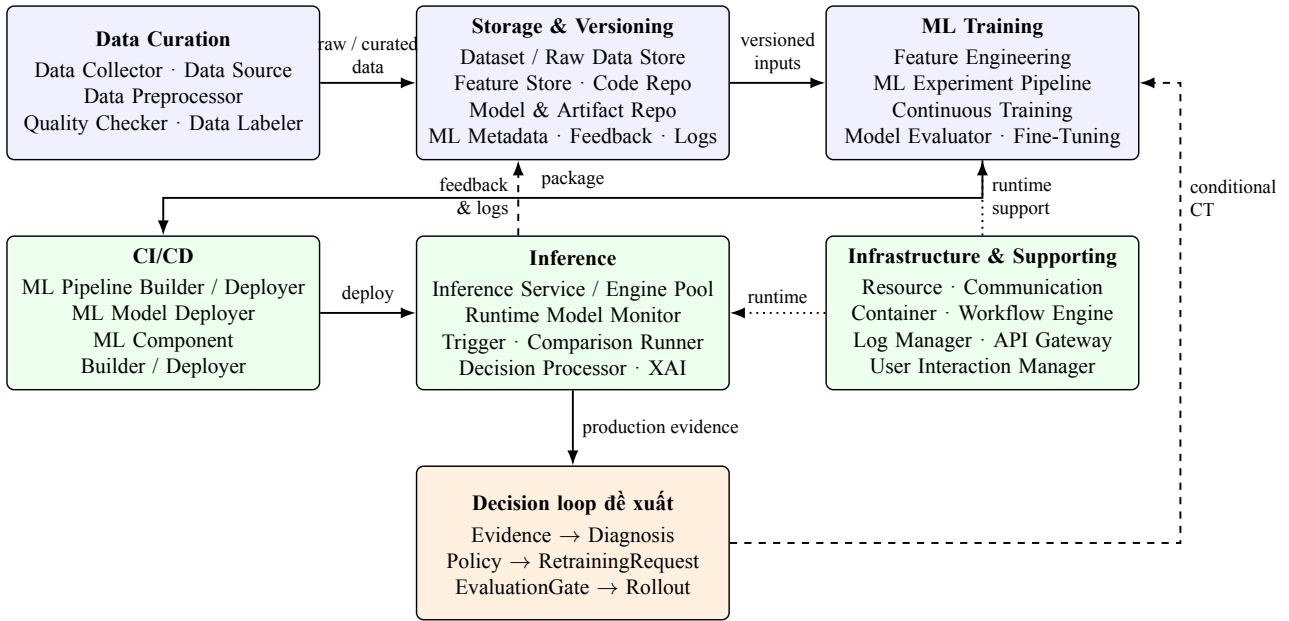


Figure 1: Bản đồ sáu nhóm component của paper và decision loop đề xuất cho repo. Màu xanh biểu diễn các nhóm nền tảng; màu cam là lớp cần xây dựng để chuyển từ V1 sang continuous training.

trầm trọng hóa sai số mô hình.

5 Hướng phát triển: Degradation-Aware Adaptive Model Maintenance

5.1 Luận đề nghiên cứu và Phạm vi

Luận đề nghiên cứu cốt lõi:

“Một hệ thống Continuous Training tin cậy phải xác định chính xác nguyên nhân suy giảm và mức độ rủi ro trước khi quyết định hành động; không tự động tái huấn luyện theo quán tính chỉ dựa trên một cảnh báo đơn lẻ.”

Vòng lặp khép kín mục tiêu:

Inference + Feedback → EvidenceWindow → DegradationEvent → Multi-signal Diagnosis → Policy Selection → Conditional CT → EvaluationGate → Shadow/Canary → Promotion / Rollback

Trong phạm vi đề tài nghiên cứu, hệ thống tập trung hiện thực hóa và kiểm chứng ba nhánh hành động chính:

1. **Harmful Data Drift (Trôi dạt có hại):** Khi dữ liệu đầu vào biến đổi kèm theo suy giảm hiệu năng thực tế và đã có đủ mẫu nhãn kiểm chứng → Kích hoạt **Conditional Auto CT** trong giới hạn ngân sách và chính sách an toàn.
2. **Data Quality / Pipeline Incident (Sự cố chất lượng dữ liệu):** Khi xuất hiện vi phạm schema, sai lệch miền giá trị hoặc mất tính tươi mới (freshness) → Dừng hoàn toàn tái huấn luyện, phát cảnh báo cho đội ngũ kỹ thuật (**Engineering Incident**).
3. **Unexplained Degradation / Concept Shift (Suy giảm chưa rõ nguyên nhân):** Khi hiệu năng giảm nhưng phân phối dữ liệu không đổi hoặc nhãn bị trễ → Kích hoạt cơ chế **Human-in-the-Loop (HITL)** để chuyên gia thẩm định dữ liệu trước khi hành động.

Code/feature semantics, label policy, architecture redesign và runtime outage nên là Engineering recommendation ở MVP. Configuration degradation/HPO là nhánh mở rộng sau khi ba path đầu đã có benchmark. Hình 2 biểu diễn thứ tự kiểm tra an toàn trước khi cho phép một candidate đi vào CT.

5.2 Câu hỏi nghiên cứu và Giả thuyết khoa học

Để chứng minh đóng góp học thuật vượt ra ngoài phạm vi xây dựng tính năng kỹ thuật, quá trình thực nghiệm sẽ tập trung kiểm chứng ba câu hỏi nghiên cứu cốt lõi:

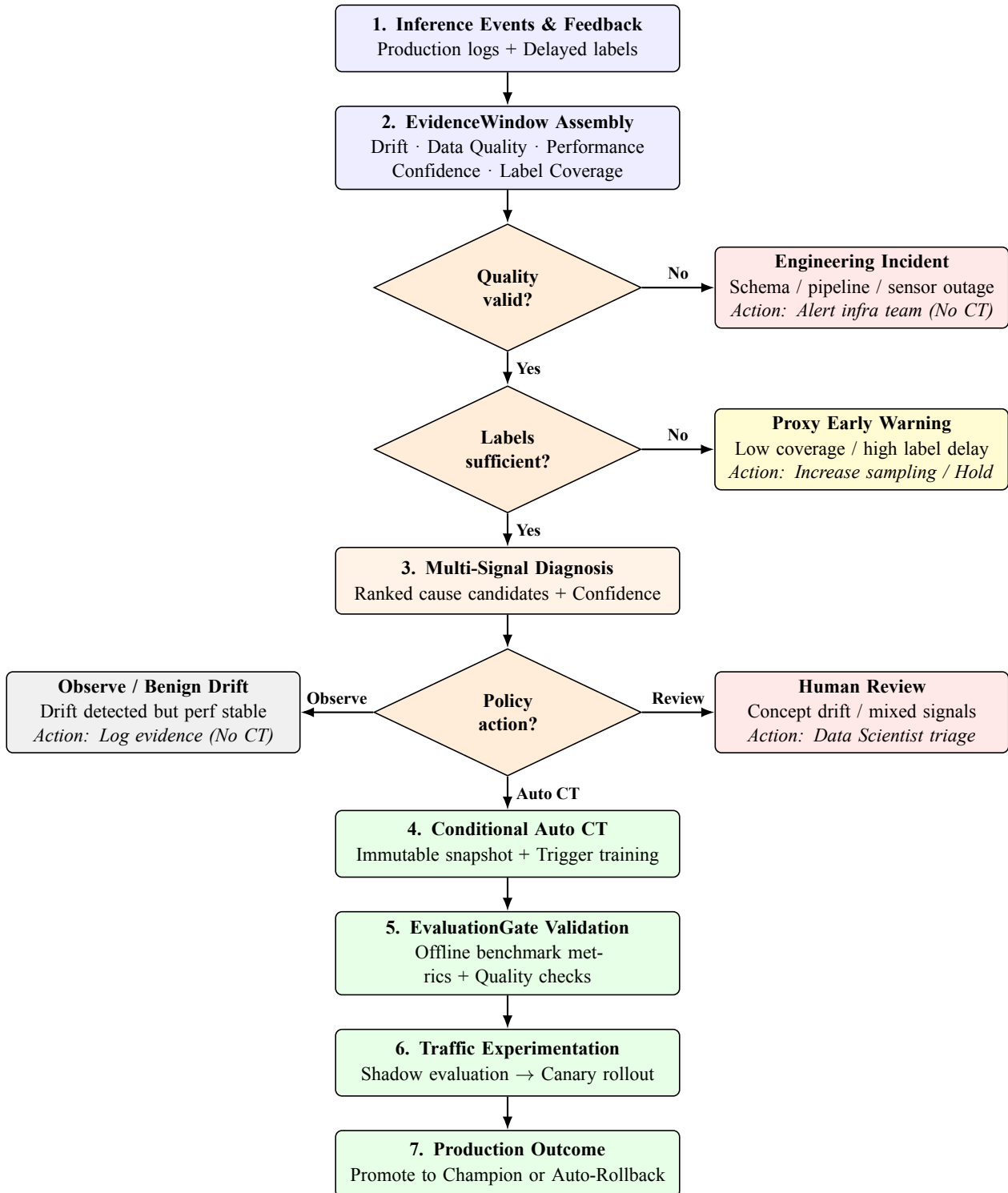


Figure 2: Workflow degradation-aware: Trục dọc thể hiện pipeline tự động hóa khép kín (happy path); các nhánh ngang thể hiện chốt an toàn và điểm can thiệp của con người/kỹ thuật khi thiếu bằng chứng.

Table 9: Hiện trạng triển khai các nhóm quy trình vòng đời then chốt [1].

Nhóm quy trình	Mức độ	Hiện trạng triển khai thực tế
Khởi tạo, đóng gói & đăng ký	Có	Upload code/data → build container image → đăng ký model version bất biến.
Thử nghiệm & huấn luyện	Một phần	Training job quản lý snapshot dữ liệu/mã nguồn; tracking metadata tích hợp qua MLflow.
Triển khai & phục vụ dự đoán	Có	Model build sẵn sàng được deploy; Gateway xác thực và proxy theo tenant/project/version.
Thu thập bằng chứng sản xuất	Một phần	Đã lưu vết features/prediction; cân bổ sung confidence, latency, prediction ID và feedback.
Phát hiện suy giảm (Detection)	Một phần	Đã có data drift và metrics hạ tầng; chưa chuẩn hóa prediction drift, data quality và performance.
Chẩn đoán nguyên nhân (Diagnosis)	Thiếu	Chưa có module suy luận nguyên nhân (cause candidates), đồ thị bằng chứng và giải thích.
Lựa chọn hành động & phê duyệt	Thiếu	Alert chưa chuyển đổi thành RetrainingRequest; thiếu cơ chế định tuyến Auto/Human/Engineering.
Tái huấn luyện, kiểm thử & thăng cấp	Một phần	Thực hiện thủ công; thiếu Evaluation Gate tự động, shadow/canary routing và auto-rollback.

- RQ1 (Hiệu quả chi phí và phục hồi):** Cơ chế quyết định hành động dựa trên bằng chứng (Evidence-to-Action Policy) có thể giảm thiểu bao nhiêu phần trăm số lần tái huấn luyện không cần thiết (unnecessary CT) so với các baseline truyền thống (Manual, Periodic, Alert-driven, Always-retrain) mà vẫn duy trì năng lực phục hồi độ chính xác tương đương khi xuất hiện trôi dạt có hại (harmful drift)?
- RQ2 (An toàn tự động hóa):** Việc kết hợp chẩn đoán nguyên nhân đa tín hiệu (Multi-Signal Diagnosis) với cơ chế xử lý nhãn trễ (Delayed-Label Handling) có thể triệt tiêu hiện tượng tự động hóa mất an toàn (unsafe automation—tái huấn luyện/thăng cấp sai lầm trên dữ liệu lỗi hoặc khi chưa đủ bằng chứng thống kê) hay không?
- RQ3 (An toàn triển khai thực tế):** Cổng kiểm thử đối kháng (Evaluation Gate) kết hợp với cơ chế so sánh lưu lượng nhân bản (Shadow Comparison) nâng cao tính an toàn và giảm thiểu rủi ro vận hành như thế nào so với quy trình huấn luyện và hoán đổi mô hình trực tiếp (direct promotion)?

Các giả thuyết khoa học tương ứng được đặt ra:

- H1:** Chính sách thích ứng giúp triệt tiêu trên 90% số lần kích hoạt CT không cần thiết trong kịch bản trôi dạt lành tính (Benign Drift - S1).
- H2:** Khối chẩn đoán kết hợp với cổng dữ liệu (Data Quality Gate) và vòng lặp chuyên gia (HITL) ngăn chặn hoàn toàn 100% các ca tái huấn luyện sai lầm khi xảy ra sự cố dữ liệu (S3) hoặc thay đổi khái niệm phức tạp (S4, S9).
- H3:** Cơ chế kiểm định đối kháng đa tầng (Offline Gate → Shadow Mode) ngăn chặn triệt để các trường hợp thoái hóa ngầm (false promotion) mà không làm tăng thời gian phục hồi (time-to-recovery) vượt quá ngưỡng cho phép.

5.3 Phân loại chính sách Bằng chứng – Hành động (Evidence-to-Action Taxonomy)

5.4 Thuật toán chẩn đoán nguyên nhân suy giảm (Diagnosis Algorithm)

Khối chẩn đoán nguyên nhân đóng vai trò hạt nhân trong việc ra quyết định. Dưới đây là đặc tả thuật toán chẩn đoán dựa trên tập luật có thể giải thích được (Interpretable Rule-based Cause Scoring) cho phiên bản MVP:

Table 10: Bảng phân loại chính sách ra quyết định thích ứng.

Mẫu hình bằng chứng	Chẩn đoán suy luận ưu tiên	Hành động an toàn đề xuất
d_{drift} cao; Quality đạt; Nhãn đủ; F1/Recall suy giảm	Trôi dạt phân phối có hại (HARMFUL_DRIFT)	Kích hoạt Conditional Auto CT trên cửa sổ dữ liệu mới (chỉ khi vượt qua kiểm tra ngân sách và công đánh giá).
Vi phạm Quality (lỗi schema, out-of-range, mất freshness)	Sự cố đường ống dữ liệu (DATA_QUALITY_ISSUE)	Engineering Incident ; đình chỉ hoàn toàn CT để tránh huấn luyện trên dữ liệu rác.
F1/Recall giảm; d_{drift} thấp hoặc mâu thuẫn	Suy giảm chưa rõ nguyên nhân (UNEXPLAINED_DEGRADATION)	Human Review ; chuyển giao chuyên gia ML phân tích lát cắt (slice analysis) và kiểm tra nhãn.
Điểm tự tin (Confidence) giảm; chưa có nhãn thực tế	Cảnh báo sớm trôi dạt (CALIBRATION_WARNING)	Tăng tần suất lấy mẫu giám sát; không tự động CT chỉ dựa trên điểm tự tin suy luận.
Độ trễ (Latency) p95 hoặc tỷ lệ lỗi 5xx tăng cao	Sự cố hạ tầng phục vụ (SERVING_INFRA_ISSUE)	Kích hoạt Auto-scaling hoặc Rollback về phiên bản trước; không can thiệp vào mô hình.
d_{drift} cao nhưng hiệu năng F1 ổn định (đủ nhãn)	Trôi dạt lành tính (BENIGN_DRIFT)	Observe / Log evidence ; ghi nhận bằng chứng và duy trì mô hình hiện tại để tiết kiệm chi phí.

Thuật toán 1: Chẩn đoán nguyên nhân suy giảm và đề xuất hành động

Đầu vào (Input): Thước kẻ EvidenceWindow E gồm tập chỉ số:

$$\{d_{\text{drift}}, q_{\text{quality}}, \Delta_{\text{perf}}, \Delta_{\text{conf}}, r_{\text{labels}}, n, CI_{\text{perf}}\}$$

Quy ước: $\Delta_{\text{perf}} = \text{Metric}_{\text{current}} - \text{Metric}_{\text{baseline}}$ (đối với metric càng cao càng tốt như F1, $\Delta_{\text{perf}} < 0$ biểu thị suy giảm).

Bước 1 (Kiểm định chất lượng dữ liệu - Data Quality Gate):

If $q_{\text{quality}} < \theta_{\text{quality}}$ **then**

Return Diagnosis(DATA_QUALITY_ISSUE, Action: ENGINEERING, Confidence: High)

Bước 2 (Kiểm định trôi dạt có hại - Harmful Drift Gate):

If $d_{\text{drift}} > \theta_{\text{drift}}$ **and** $\Delta_{\text{perf}} < \theta_{\text{perf}}$ **and** $r_{\text{labels}} \geq r_{\text{min}}$ **and** $n \geq n_{\text{min}}$ **and** Cận dưới của CI_{perf} xác nhận suy giảm có ý nghĩa thống kê **then**

If Kiểm tra điều kiện (Cooldown OK $\wedge$ Budget OK) **then**

Return Diagnosis(HARMFUL_DRIFT, Action: AUTO_CT, Confidence: High)

Else

Return Diagnosis(HARMFUL_DRIFT, Action: HUMAN_REVIEW, Confidence: High)

Bước 3 (Suy giảm hiệu năng chưa rõ nguyên nhân):

If $\Delta_{\text{perf}} < \theta_{\text{perf}}$ **and** $d_{\text{drift}} \leq \theta_{\text{drift}}$ **then**

Return Diagnosis(UNEXPLAINED_DEGRADATION, Action: HUMAN_REVIEW, Confidence: Medium)

Bước 4 (Trôi dạt lành tính - Benign Drift):

If $d_{\text{drift}} > \theta_{\text{drift}}$ **and** $|\Delta_{\text{perf}}| \approx 0$ **and** $r_{\text{labels}} \geq r_{\text{min}}$ **then**

Return Diagnosis(BENIGN_DRIFT, Action: OBSERVE, Confidence: High)

Bước 5 (Cảnh báo sớm từ phân phối điểm tự tin):

If Δ_{conf} lệch chuẩn có ý nghĩa **and** $r_{\text{labels}} < r_{\text{min}}$ **then**

Return Diagnosis(CALIBRATION_WARNING, Action: INCREASE_SAMPLING, Confidence: Low)

Mặc định (Default Fallback):

Return Diagnosis(AMBIGUOUS, Action: HUMAN_REVIEW, Confidence: Low)

5.5 Chiến lược xử lý nhãn trễ (Delayed-Label Handling Strategy)

Nhãn thực tế (ground-truth labels) thường có độ trễ lớn so với thời điểm suy luận, tạo nên điểm nghẽn nghiêm trọng nhất trong việc giám sát hiệu năng mô hình. Hệ thống đề xuất 5 nguyên tắc xử lý nhãn trễ:

1. **Chuỗi liên kết mã định danh dự đoán (End-to-end Prediction ID):** Gateway model-server tạo mã định danh duy nhất (`prediction_id`) và trả về cho client trong HTTP response, đồng thời gắn kèm mã này vào event stream lưu trữ. Client sử dụng chính `prediction_id` để gửi nhãn thực tế qua Feedback API. Đây là **điều kiện tiên quyết** để thực hiện phép join giữa dự đoán và nhãn thực tế.
2. **Cửa sổ trượt gắn mốc thời gian nhãn (Watermark-based Sliding Window):** Phép tính toán hiệu năng (F1, Precision, Recall) chỉ được thực hiện trên cửa sổ thời gian $[t - W, t]$ khi tỷ lệ phủ nhãn đạt ngưỡng tin cậy tối thiểu:

$$r_{\text{labels}} = \frac{|\text{Dự đoán đã có nhãn}|}{|\text{Tổng số lượt dự đoán}|} \geq r_{\text{min}}$$

Nếu $r_{\text{labels}} < r_{\text{min}}$, hệ thống ghi nhận trạng thái chưa đủ nhãn (INSUFFICIENT_LABELS) và tuyệt đối không nội suy hiệu năng giả định.

3. **Chỉ số đại diện khi thiếu nhãn (Proxy Signals):** Khi nhãn chưa sẵn sàng, hệ thống dựa vào các tín hiệu đại diện: độ trôi dạt phân phối đầu ra $P(\hat{Y})$, độ dịch chuyển điểm tự tin (calibrated confidence drift) và biến động độ trễ để phát cảnh báo sớm, không dùng để kích hoạt CT trực tiếp.
4. **Phân tầng độ tin cậy của nhãn (Label Trust Levels):** Mỗi bản ghi nhãn phản hồi đều lưu kèm nguồn thu thập (`label_source`: người dùng cuối, chuyên gia gán nhãn, hay pipeline tự động) và mức độ tin cậy (`trust_level`), ngăn ngừa việc trộn lẫn nhãn nhiễu vào dữ liệu huấn luyện mới.
5. **Ước lượng thống kê với độ phủ nhãn một phần:** Khi chỉ có một phần dự đoán được gán nhãn (ví dụ 30%), hiệu năng được tính toán kèm khoảng tin cậy 95% (CI_{perf}). Công quyết định chỉ kích hoạt CT khi cận dưới của khoảng tin cậy xác nhận sự suy giảm thực sự.

5.6 Mô hình dữ liệu trạng thái (Minimal State Model)

Để hiện thực hóa vòng lặp khép kín, Control Plane cần bổ sung 8 thực thể dữ liệu bất biến:

Table 11: Các thực thể dữ liệu cốt lõi bổ sung vào Control Plane.

Thực thể (Entity)	Thuộc tính then chốt	Mục đích kiến trúc & Nghiệp vụ
PredictionRecord	<code>prediction_id</code> , <code>request_id</code> , <code>tenant</code> , <code>project</code> , <code>version</code> , <code>timestamp</code> , <code>features</code> , <code>prediction</code> , <code>confidence</code> , <code>latency</code> , <code>status</code>	Ghi nhận từng giao dịch suy luận và thiết lập khóa liên kết (correlation key) cho dữ liệu phản hồi.
Feedback	<code>feedback_id</code> , <code>prediction_id</code> , <code>ground_truth</code> , <code>label_timestamp</code> , <code>label_source</code> , <code>trust_level</code> , <code>correction_version</code>	Lưu vết nhãn thực tế trả về từ client; hỗ trợ nhiều lượt phản hồi và đính chính nhãn (correction).
EvidenceWindow	<code>model_version</code> , <code>window_start</code> , <code>window_end</code> , <code>baseline_ref</code> , <code>metric_values</code> (drift, quality, perf, conf), <code>slices</code> , <code>label_coverage</code> , <code>policy_version</code>	Lưu trữ bằng chứng vận hành bất biến theo cửa sổ thời gian phục vụ chẩn đoán và kiểm toán.
DegradationEvent	<code>evidence_window_ids</code> , <code>severity</code> , <code>affected_component</code> , <code>deduplication_key</code> , <code>status</code> , <code>created_at</code>	Chuẩn hóa sự kiện cảnh báo suy giảm với tính chất bất biến và chống trùng lặp (idempotency).
Diagnosis	<code>degradation_event_id</code> , <code>ranked_causes</code> (JSON), <code>top_confidence</code> , <code>explanation</code> , <code>lineage_snapshot</code> , <code>policy_version</code>	Tách biệt tường minh khâu suy luận nguyên nhân gốc rễ khỏi khâu phát hiện cảnh báo thô.

Thực thể (Entity)	Thuộc tính then chốt	Mục đích kiến trúc & Nghiệp vụ
RetrainingRequest	diagnosis_id, strategy (auto/human/eng), data_snapshot_uri, config_version, budget_limit, cooldown_until, status, approved_by	Đơn vị điều phối vòng đời tái huấn luyện thay thế cho việc gọi trực tiếp training job từ drift run.
EvaluationGate	candidate_version, champion_version, offline_metrics, calibration_score, latency_p95, error_rate, data_quality_passed, decision (pass/fail/hold)	Cổng thẩm định đối kháng nghiêm ngặt trước khi cho phép mô hình ứng viên tiếp cận môi trường sản xuất.
TrafficExperiment	experiment_type (shadow/canary), candidate_version, champion_version, hash_seed, weight_schedule, guardrails, current_weight, rollback_reason	Điều phối và ghi nhận toàn bộ quá trình thử nghiệm lưu lượng thực tế và cơ chế thu hồi tự động.

Một prediction có thể chưa có label hoặc có nhiều lần label/correction; vì vậy PredictionRecord và Feedback nên là hai bảng có quan hệ một-nhiều, kèm unique/idempotency constraint cho nguồn feedback và version correction.

6 Ranh giới tự động hoá và Quy trình kiểm thử thăng cấp

Table 12: Ma trận phân định ranh giới tự động hóa (Auto vs. Human vs. Engineering).

Cấp độ tự động	Quyền hạn của hệ thống	Ranh giới cấm tự động (Cần con người/CI)
Observe / No Action	Ghi nhận sự kiện, tăng tỷ lệ lấy mẫu, theo dõi cửa sổ tiếp theo.	Tuyệt đối không kích hoạt huấn luyện (tránh training storm).
Conditional Auto CT	Snapshot dữ liệu được phê duyệt, kích hoạt huấn luyện, kiểm thử qua EvaluationGate và đăng ký candidate.	Không tự ý thay đổi định nghĩa nhãn, cấu trúc feature, mã nguồn hay kiến trúc mô hình.
Human-approved CT	Chuẩn bị đầy đủ báo cáo chẩn đoán, vết nguồn gốc (lineage) và kế hoạch huấn luyện ứng viên.	Không tự quyết định đánh đổi rủi ro kinh doanh hoặc tiếp tục khi nhãn mâu thuẫn.
Engineering Incident	Tạo issue cảnh báo kèm bằng chứng audit trail, chuyển giao đội ngũ kỹ thuật.	Không tự can thiệp chỉnh sửa pipeline, dependency thư viện, mã ETL hay hạ tầng runtime.

6.1 Kiến trúc thử nghiệm lưu lượng thực tế (Shadow & Canary Architecture)

Quy trình thăng cấp (promotion) trong kiến trúc đề xuất tuân thủ phương pháp kiểm định đa tầng lũy tiến thay vì hoán đổi trực tiếp sau huấn luyện:

- 1. Chế độ nhân bản lưu lượng (Shadow Mode - Phase 5a):** Mô hình ứng viên (Candidate) nhận một bản sao lưu lượng thực tế từ gateway mà không làm ảnh hưởng tới phản hồi trả về người dùng (Champion vẫn trả response). Dự đoán của candidate được ghi nhận độc lập vào `paas_shadow_logs` để phục vụ so sánh đối kháng trên cùng request ID.

2. **Chế độ lưu lượng tăng dần (Canary Rollout - Phase 5b):** Định tuyến lưu lượng thực tế dựa trên băm nhất quán (consistent hashing) theo khóa phân nhóm ổn định (User/Session ID). Tỷ lệ lưu lượng được nâng dần theo các mốc 5% → 10% → 25% → 50% → 100% khi và chỉ khi thỏa mãn đầy đủ các chỉ số rào chắn an toàn (Guardrails).
3. **Tiêu chí rào chắn an toàn (Guardrail Evaluation):** Đánh giá liên tục trên cửa sổ trượt có kiểm định thống kê:
 - Tỷ lệ lỗi (Error rate) của Candidate $\leq$ Champion $+\epsilon$
 - Độ trễ p95 của Candidate $\leq$ Champion $\times 1.2$
 - Khoảng cách phân phối dự đoán (KL-Divergence) $\leq \theta_{\text{dist}}$
 - Hiệu năng thực tế (khi có nhãn) của Candidate $\geq$ Champion $-\delta$
4. **Thu hồi tự động (Automated Rollback):** Tự động ngắt toàn bộ lưu lượng của Candidate và chuyển đổi Blue-Green về Champion cũ khi vi phạm guardrail trong k cửa sổ liên tiếp, đồng thời lưu vết rollback_reason vào thực thể TrafficExperiment.

Hình 3 mô tả trực quan quy trình kiểm định thăng cấp lũy tiến này.

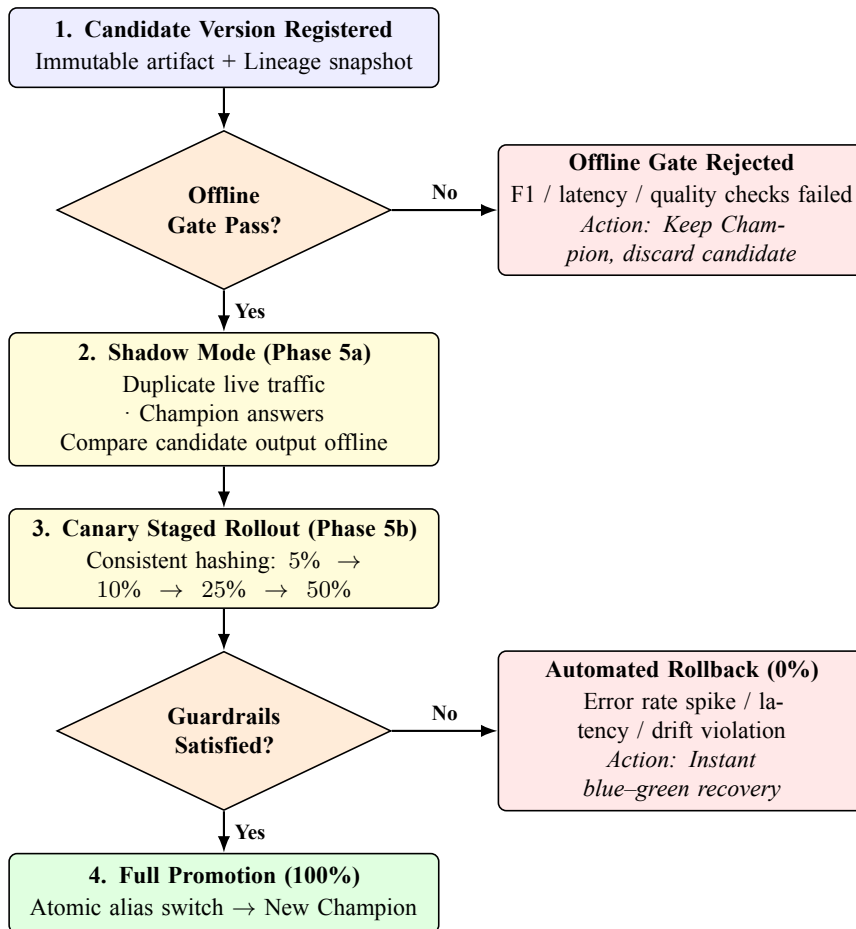


Figure 3: Protocol champion-challenger đề xuất. Trực dọc thể hiện các tầng kiểm thử lũy tiến (Offline Gate → Shadow → Canary → Full Promotion); nhánh ngang thể hiện điểm dừng an toàn và cơ chế rollback tự động.

7 Lộ trình hiện thực hoá và Kế hoạch nghiên cứu

Lộ trình thực hiện được thiết kế kéo dài 26 tuần, phân bổ thành 4 giai đoạn logic từ chuẩn bị thực nghiệm đến triển khai hoàn thiện và công bố:

Table 13: Lộ trình nghiên cứu và phát triển 26 tuần.

Giai đoạn	Mục tiêu trọng tâm	Sản phẩm bàn giao (Deliverables)	Tiêu chí hoàn thành (Exit Criteria)
— Giai đoạn 1: Chuẩn bị & Chứng minh khái niệm (PoC) —			
Phase 0 (Tuần 1–2)	Đóng băng giao thức nghiên cứu	Bộ dataset tabular, framework tạo kịch bản trôi dạt (scenario injection), danh sách baseline, ngân sách và hạt giống ngẫu nhiên (seeds).	Giao thức benchmark được kiểm soát phiên bản qua Git và S3 manifest trước khi thực hiện đo đạc.
Phase 0.5 (Tuần 3–4)	Thử nghiệm lát cắt dọc (Vertical Slice PoC)	Tích hợp khép kín trên 1 kịch bản mẫu (S2 Harmful Drift): Inject drift → Detect → Hardcoded Diagnosis → Retrain → Evaluate → Promote.	Xác nhận tính thông suốt của luồng dữ liệu và phát hiện sớm các điểm nghẽn tích hợp giữa các service.
— Giai đoạn 2: Xây dựng nền tảng đo lường & Giám sát —			
Phase 1 (Tuần 5–9)	Hoàn thiện hạ tầng dữ liệu & nhận trả	1a (Tuần 5–6): Hoàn chỉnh event contract gửi prediction_id, confidence, latency, status. 1b (Tuần 7–8): Xây dựng Feedback API + Store, thực thể PredictionRecord, kiểm thử phép join nhẵn. 1c (Tuần 9): Snapshot dữ liệu bất biến, thực thể EvidenceWindow, kiểm tra chất lượng dữ liệu cơ bản.	Thực hiện thành công phép join prediction–feedback trên dữ liệu mẫu; truy vết trọn vẹn từ model → data → code.
Phase 2 (Tuần 10–13)	Mở rộng phát hiện đa tín hiệu (Detection)	2a (Tuần 10–11): Mở rộng Evidently đo prediction drift, data quality checks, slice monitoring. 2b (Tuần 12): Giám sát hiệu năng F1/Recall trên cửa sổ trượt watermark khi $r_{\text{labels}} \geq r_{\text{min}}$. 2c (Tuần 13): Thực thể DegradationEvent chống trùng lặp, kiểm thử phát hiện trên kịch bản S0–S3.	Phân loại chính xác các dạng bất thường (S0, S1, S2, S3); đo lường được Precision/Recall của khâu phát hiện.
— Giai đoạn 3: Trí tuệ ra quyết định & Vòng đời CT —			

Giai đoạn	Mục tiêu trọng tâm	Sản phẩm bàn giao (Deliverables)	Tiêu chí hoàn thành (Exit Criteria)
Phase 3 (Tuần 14–17)	Chẩn đoán nguyên nhân & Vòng lặp HITL	3a (Tuần 14–15): Thực thể <code>Diagnosis</code> , bộ quy luật tính điểm nguyên nhân (Thuật toán 1), văn bản giải thích. 3b (Tuần 16): Thực thể <code>RetrainingRequest</code> , chính sách cooldown, API phê duyệt. 3c (Tuần 17): Giao diện duyệt yêu cầu (Approval Inbox), lưu vết kiểm toán (Audit Trail).	Kiểm thử tự động chứng minh cảnh báo không tự ý gọi huấn luyện; độ chính xác chẩn đoán đạt trên kịch bản S0–S4, S9.
Phase 4 (Tuần 18–20)	Tái huấn luyện ứng viên & Công kiểm thử	Trình đóng gói dữ liệu cửa sổ mới, huấn luyện tự động, thực thể <code>EvaluationGate</code> , đăng ký candidate trong Model Registry.	Candidate có lineage bất biến và vượt qua công kiểm định offline trước khi được xem xét triển khai.
Phase 5 (Tuần 21–23)	Thử nghiệm so sánh môi trường thực tế	5a (Tuần 21–22): Điều phối Shadow Mode tại gateway, lưu log <code>paas_shadow_logs</code> , đo đặc sai lệch. 5b (Tuần 23, Stretch): Canary weighted routing, rào chắn guardrails và cơ chế tự động rollback.	Hệ thống hỗ trợ thử nghiệm đối kháng Champion–Candidate an toàn trên môi trường production.
— Giai đoạn 4: Thử nghiệm tổng thể & Đóng gói công bố —			
Phase 6 (Tuần 24–26)	Đánh giá Benchmark & Hoàn thiện bài báo	Chạy toàn bộ kịch bản thực nghiệm với 6 baseline, phân tích ablation, độ nhạy (sensitivity), chi phí tài nguyên và hoàn thiện bài báo khoa học.	Kết quả có thể tái lập hoàn toàn; bộ dữ liệu, mã nguồn và báo cáo kỹ thuật được đóng gói chuẩn mực.

7.1 Phân bổ trách nhiệm theo phân hệ dịch vụ (Subsystem Ownership)

- **model-server + consumer:** Hoàn chỉnh event contract (gửi `prediction_id`, confidence, latency, status); consumer lưu vết bền vững vào PostgreSQL và quản lý transactional outbox.
- **control plane:** Bổ sung 8 thực thể dữ liệu mới, cung cấp Feedback API, quản lý logic chẩn đoán, chính sách phê duyệt và điều phối thử nghiệm lưu lượng.
- **evidently service:** Mở rộng từ data drift sang prediction drift, kiểm định chất lượng dữ liệu và tính toán chỉ số hiệu năng trên nhãn thực tế.
- **training-runner + execution backend:** Nhận snapshot dữ liệu bất biến, thực thi huấn luyện có giới hạn tài nguyên và trích xuất chỉ số kiểm định chuẩn hóa.
- **registry + deployment + GitOps:** Quản lý metadata candidate/champion, điều phối cấu hình định tuyến lưu lượng và giám sát rào chắn an toàn.
- **web frontend:** Xây dựng dòng thời gian bằng chứng (Evidence Timeline), hộp thư phê duyệt (Approval Inbox) và giao diện so sánh mô hình trực quan.

8 Thiết kế thực nghiệm và Đánh giá Benchmark

Để khẳng định tính đóng góp khoa học, bộ benchmark thực nghiệm phải được thiết kế với **nguyên nhân suy giảm đã biết trước (Ground-truth Cause)**, vượt ra ngoài việc chỉ quan sát nhãn bài toán máy học.

8.1 Không gian kịch bản thực nghiệm (Benchmark Scenarios)

Table 14: Danh mục các kịch bản thực nghiệm phân theo mức độ ưu tiên.

Mã	Phân tầng	Bản chất kịch bản (Scenario Injection)	Hành động kỳ vọng chuẩn xác
S0	Cốt lõi	Môi trường ổn định, không có suy giảm (Stable)	Không can thiệp (NO_ACTION)
S1	Cốt lõi	Trôi dạt lành tính: $P(X)$ đổi nhưng $P(Y X)$ ổn định	Cảnh báo, theo dõi (OBSERVE)
S2	Cốt lõi	Trôi dạt có hại: $P(X)$ đổi kéo theo F1 giảm, đủ nhãn	Tái huấn luyện tự động (AUTO_CT)
S3	Cốt lõi	Sự cố chất lượng dữ liệu: lỗi schema, range, freshness	Đình chỉ CT, mở ENGINEERING_INCIDENT
S4	Quan trọng	Dịch chuyển khái niệm/nhãn: $P(Y X)$ đổi đột ngột	Phê duyệt chuyên gia (HUMAN_APPROVED_CT)
S9	Quan trọng	Tín hiệu hỗn hợp: vừa trôi dạt dữ liệu vừa trễ nhãn	Phân tích chuyên sâu (HUMAN_REVIEW)
S5	Mở rộng	Dữ liệu huấn luyện bị lão hóa dần theo thời gian	Tái huấn luyện định kỳ hoặc có điều kiện
S6	Mở rộng	Suy giảm cấu hình siêu tham số (HPO degradation)	Cập nhật không gian tìm kiếm HPO
S7	Mở rộng	Lỗi mã nguồn xử lý đặc trưng (Feature code bug)	Sửa đổi qua luồng CI/CD kỹ thuật
S8	Mở rộng	Sự cố hạ tầng phục vụ (OOM, nghẽn mạng)	Thu hồi tự động hoặc mở rộng tài nguyên

8.2 Các phương pháp đối chứng (Baselines)

Quá trình thực nghiệm so sánh phương pháp đề xuất với 5 đường cơ sở chuẩn mực:

- **B0 – Observe-only:** Không thực hiện tái huấn luyện trong suốt chu kỳ; đo lường mức độ suy giảm tự nhiên.
- **B1 – Manual Retraining:** Tái huấn luyện theo thao tác thủ công định kỳ của chuyên gia.
- **B2 – Periodic CT:** Tái huấn luyện cố định theo lịch trình (ví dụ: hàng tuần hoặc hàng tháng).
- **B3 – Alert-driven CT:** Tự động kích hoạt tái huấn luyện ngay khi có bất kỳ cảnh báo data drift nào (baseline đại diện cho cách tiếp cận truyền thống).
- **B4 – Always Retrain:** Huấn luyện lại liên tục sau mỗi lô dữ liệu mới (đóng vai trò đường cơ sở trần về chi phí tài nguyên).
- **B5 – Degradation-Aware Adaptive CT (Đề xuất):** Chuỗi xử lý Detection → Diagnosis → Policy → Conditional CT → Evaluation Gate.

8.3 Hệ thống chỉ số đánh giá đa chiều (Evaluation Metrics)

8.4 Mô hình chi phí và Hiệu quả đầu tư (Cost-Benefit Model)

Tổng chi phí của một chu kỳ bảo trì được mô hình hóa theo công thức:

$$C_{\text{Cycle}} = C_{\text{data_prep}} + C_{\text{train}} + C_{\text{evaluate}} + C_{\text{shadow/canary}} + C_{\text{human_effort}}$$

Table 15: Khung chỉ số đánh giá thực nghiệm toàn diện.

Chiều cạnh đánh giá	Tập chỉ số đo lường chi tiết
Chất lượng phát hiện	Precision, Recall, F1 của cảnh báo; độ trễ phát hiện (thời gian từ lúc inject tới cảnh báo đầu tiên).
Chất lượng chẩn đoán	Độ chính xác Top-k và Multi-label của nguyên nhân suy giảm; sai số hiệu chuẩn độ tin cậy (ECE, Brier Score).
Chất lượng quyết định	Tỷ lệ ra quyết định chính xác (Strategy Accuracy); tỷ lệ tự động hóa mất an toàn (Unsafe Automation Rate).
Năng lực phục hồi	Thời gian phục hồi (Time-to-Recovery); đường cong phục hồi hiệu năng (F1 recovery curve).
Hiệu quả chi phí	Số lần CT không cần thiết bị triệt tiêu; tổng số giờ tính toán (GPU/CPU-hours); chi phí serving shadow/canary.
Gánh nặng con người	Số lượng yêu cầu cần duyệt; thời gian phản hồi trung bình của chuyên gia; tỷ lệ phủ quyết (Override rate).
An toàn triển khai	Tỷ lệ vi phạm guardrails; tỷ lệ thu hồi tự động thành công (Rollback success); độ tin cậy kiểm thử candidate.

Hiệu quả của chính sách thích ứng được định lượng thông qua chỉ số tỷ suất lợi ích trên chi phí tính toán (Efficiency Ratio):

$$ROI_{CT} = \frac{\Delta \text{Performance Gained} \times \text{Business Value}}{C_{\text{Cycle}} + C_{\text{Risk}}}$$

Một chu kỳ CT chỉ được coi là hợp lý khi $ROI_{CT} > 1$, khẳng định tính kinh tế của việc chất lọc điều kiện tái huấn luyện.

9 Quản trị rủi ro và Ranh giới an toàn hệ thống

- **Thách thức về nhãn trễ (Delayed Labels):** Hệ thống áp dụng nguyên tắc phân tách ranh giới giữa giám sát tín hiệu đại diện (Proxy Monitoring) và tính toán hiệu năng đã xác thực (Confirmed Performance); chỉ kích hoạt CT khi độ phủ nhãn r_{labels} đạt ý nghĩa thống kê (chi tiết tại Mục 5.5).
- **Tính mơ hồ trong quan hệ nhân quả (Causal Ambiguity):** Khi xảy ra đồng thời nhiều dạng biến động, khối chẩn đoán xếp hạng các nguyên nhân theo xác suất và tự động rơi về chế độ an toàn HUMAN_REVIEW, tránh việc suy diễn nguyên nhân tuyệt đối khi thiếu chứng cứ.
- **Ngăn ngừa bão tái huấn luyện (Training Storm):** Thiết lập thời gian giãn cách tối thiểu (Cooldown Period) giữa hai lần CT liên tiếp của cùng một mô hình và đặt ngưỡng trần ngân sách tính toán (Budget Cap) theo tuần/tháng.
- **Bảo mật và cô lập đa tenant (Multi-Tenancy Isolation):** Mọi bản ghi sự kiện, dữ liệu phản hồi và truy vấn bằng chứng đều được gắn khóa phân vùng cứng theo `tenant_id`, tuân thủ chính sách lưu giữ và xóa dữ liệu định kỳ.
- **Kiểm soát sai số tổng quát hóa (External Validity):** Các kịch bản thử nghiệm được tạo với nhiều mức cường độ, kết hợp dữ liệu nhiều và độ trễ nhãn ngẫu nhiên trên nhiều hạt giống (seeds) để bảo đảm kết luận thực nghiệm mang tính khách quan.

10 Tổng quan các công trình liên quan (Related Work)

10.1 Khung kiến trúc MLOps và Mô hình trưởng thành

Nghiên cứu của Amou Najafabadi *et al.* [1] là công trình systematic mapping toàn diện nhất hiện nay về cấu trúc và quy trình MLOps. Mô hình trưởng thành của Google Cloud [3] định nghĩa 3 cấp độ (ML0–ML2) với Continuous Training nằm ở mức trưởng thành cao nhất ML2. Mô hình 5 cấp độ của Microsoft [4] và khảo sát của Bayram & Ahmed [5] nhấn mạnh tính vững chắc (robustness) và quản trị rủi ro trong môi trường sản xuất.

10.2 Giám sát mô hình và Phát hiện trôi dạt

Thư viện mã nguồn mở Evidently AI [6] cung cấp khả năng đo lường data drift và kiểm tra chất lượng dữ liệu cơ bản. NannyML [7] tập trung vào phương pháp ước lượng hiệu năng khi chưa có nhãn. Về mặt thuật toán học thuật, các cơ chế phát hiện trôi dạt kinh điển bao gồm ADWIN [8], Page-Hinkley test, DDM [9] và tổng quan toàn diện về concept drift của Lu *et al.* [10].

10.3 Hệ thống Continuous Training và Tự thích ứng

Luo *et al.* [11] đề xuất kiến trúc thử nghiệm thích ứng tự động trên nền tảng nguồn mở. Semola *et al.* [12] giới thiệu mô hình Continual Learning as a Service (CLaaS). Trong khi đa số các công trình hiện nay chỉ tập trung vào câu hỏi “*Khi nào nên huấn luyện lại?*” (Triggering), đề tài này đóng góp vào khoảng trống “*Tại sao xảy ra suy giảm và nên lựa chọn hành động an toàn nào?*” (Multi-Signal Diagnosis → Safe Action Selection).

11 Kết luận và Đóng góp khoa học

Quá trình đối chiếu 39 thành phần kiến trúc và 56 bước quy trình khẳng định repository MLOps PaaS đã sở hữu nền tảng vững chắc về quản trị vòng đời mô hình bất biến, hạ tầng huấn luyện đa dạng và kênh thu thập sự kiện suy luận. Đóng góp cốt lõi tiếp theo của đề tài không phải là việc gắn thêm một trigger tái huấn luyện cơ học, mà là xây dựng khối **trí tuệ ra quyết định thích ứng (Adaptive Decision Intelligence)** giúp hệ thống nhận thức sâu sắc nguyên nhân suy giảm trước khi hành động.

Ba sản phẩm khoa học chính (Deliverables) được đóng gói bao gồm:

1. **Khung phân loại & Thuật toán chẩn đoán (Taxonomy & Algorithm):** Hệ thống phân loại Evidence-to-Action và thuật toán chẩn đoán nguyên nhân suy giảm có thể giải thích được, tích hợp cơ chế xử lý nhãn trễ.
2. **Nền tảng thực nghiệm nguồn mở (Open Platform):** Bản hiện thực hoàn chỉnh trên nền tảng MLOps PaaS, chứng minh tính khả thi của chu trình khép kín từ Detection → Diagnosis → Conditional CT → Progressive Promotion.
3. **Bộ Benchmark & Đánh giá thực nghiệm (Benchmark Protocol):** Bộ dữ liệu thực nghiệm chuẩn hóa với nguyên nhân suy giảm đã biết trước, kiểm chứng định lượng vượt trội của phương pháp đề xuất về hiệu quả chi phí, độ chính xác phục hồi và tính an toàn vận hành.

References

- [1] F. Amou Najafabadi, J. Bogner, I. Gerostathopoulos, and P. Lago, “An architectural perspective on MLOps: Structures, processes, tools, and stakeholders,” *Information and Software Technology*, vol. 193, Art. 108029, 2026. doi: [10.1016/j.infsof.2026.108029](https://doi.org/10.1016/j.infsof.2026.108029).
- [2] MLOps PaaS repository, source inspection of control plane, model server, consumer, Evidently, training, registry, deployment, GitOps manifests, and architecture documentation, accessed August 28, 2026.
- [3] K. Salama, J. Kazmierczak, and D. Schut, “Practitioners guide to MLOps: A framework for continuous delivery and automation of machine learning,” Google Cloud, 2021.
- [4] Microsoft, “MLOps maturity model,” Microsoft Learn, 2025. <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/mlops-maturity-model>.
- [5] F. Bayram and B. S. Ahmed, “Towards trustworthy machine learning in production: An overview of the robustness in MLOps approach,” *ACM Computing Surveys*, vol. 57, no. 5, pp. 1–35, 2025.
- [6] Evidently AI, “Evidently: Open-source ML monitoring,” <https://www.evidentlyai.com/>.
- [7] NannyML, “NannyML: Performance estimation and monitoring,” <https://www.nannyml.com/>.
- [8] A. Bifet and R. Gavaldà, “Learning from time-changing data with adaptive windowing,” in *Proc. SDM*, SIAM, 2007, pp. 443–448.
- [9] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, “Learning with drift detection,” in *SBLA*, Springer, 2004, pp. 286–295.
- [10] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, “Learning under concept drift: A review,” *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, 2019.
- [11] Y. Luo, M. Raatikainen, and J. K. Nurminen, “Autonomously adaptive machine learning systems: Experimentation-driven open-source pipeline,” 2023.
- [12] R. Semola, V. Lomonaco, and D. Bacciu, “Continual-learning-as-a-service (CLaaS): On-demand efficient adaptation of predictive models,” 2023.